ITA CHALLENGE SPRINT · SEMANA 3 · PROTÓTIPO V1 E ARQUITETURA

# O protótipo já roda — e processa a base real do desafio.

GerencIA V1: a arquitetura de ponta a ponta, o modelo de IA e sua justificativa, o fluxo de dados com retroalimentação, segurança e privacidade — e as limitações técnicas que já encontramos construindo.

Base oficial: 650 eventos · 14 regras · 15/15 casos do gabarito reproduzidos · 98,8% de concordância de nível · dados fictícios (declarado)

---

Primeiro protótipo

## O que já existe, rodando

**Não é mockup.** O V1 lê a planilha oficial, aplica as 14 regras, audita a rotulagem, monta o painel e recebe capturas ao vivo das estações — tudo com um comando. Reproduz **15/15** dos casos do gabarito e concorda com o nível de risco da base em **98,8%**; os dois motores (Python e o do navegador) batem em tudo.

Rodar o protótipo inteiro são quatro comandos — sem instalar dependências (Python puro):

| | |
|---|---|
| `python3 run.py gerar` | lê a planilha, aplica as regras, audita, monta o painel (~2 s) |
| `python3 run.py coletor` | recebe as capturas do espia-borda e regenera o painel a cada uma |
| `python3 testes.py` | 37 verificações de sanidade |
| `python3 testes_paridade.py` | confere o motor Python contra o motor do navegador |

Funcionalidades principais

## O que já pode ser demonstrado

### Rastrear (busca reversa)

Parte de uma informação e mostra ferramentas, pessoas e vezes — a pergunta norteadora do desafio.

### Motor de risco R01–R14

Decide o nível por precedência declarada e devolve a cadeia de evidência de cada evento.

### Auditoria

Indicadores recalculados × declarados e os seis achados de rotulagem.

### Validação

Os 15 casos oficiais, reproduzidos — 15/15, verde.

### Alertas

Fila por nível de risco, com filtros por área, usuário, ferramenta, tipo e período.

### Captura ao vivo

Extensão (conteúdo), agente (metadado) e proxy (conteúdo), unificados ao legado, marcados pela origem.

---

Critério 01 · Modelo de IA

## Qual modelo, e por quê

A solução usa **IA em duas camadas**, em cascata — barata e explicável primeiro, flexível e cara só quando necessário:

Camada 1 · decide

### IA clássica (motor próprio)

Regras R01–R14 com precedência + detectores determinísticos (CPF/CNPJ por dígito verificador, cartão por Luhn, credenciais por padrão) + fingerprint por MinHash/SimHash/shingles para reconhecer trechos de documentos do acervo. Determinística, explicável e barata.

Camada 2 · complementa

### LLM — Claude (claude-opus-5)

Entra em cascata só quando a camada 1 não resolve: compreende sinônimo e paráfrase no rastreio e na classificação. Devolve aprendizado para a camada 1, reduzindo o uso (e o custo) da LLM ao longo do tempo.

### Justificativa

Governança exige o **porquê** auditável de cada decisão de risco — isso pede regras explícitas, não uma caixa-preta. Mas o problema é linguagem natural (a pergunta do auditor, o conteúdo dos prompts), que pede um LLM. A cascata concilia os dois: a IA clássica resolve a maioria com evidência e custo zero de inferência; o LLM cobre o resíduo ambíguo e **ensina** a camada clássica, de modo que o sistema fica mais barato e mais preciso com o uso.

Critério 02 · Arquitetura

## Componentes e integrações




Implantação on-premise: núcleo numa **VLAN sem saída** (docker-compose), com um **gateway de notificação na DMZ** como única peça com saída (WhatsApp/e-mail); firewall, systemd e segredos fora do repositório.

Critério 03 · Fluxo de dados

## Entrada, processamento e retroalimentação

### De onde vêm e como são processados

- **Entrada (legado):** a planilha oficial, lida por `xlsx_min` → modelo interno.
- **Entrada (presente):** capturas da borda via `POST /eventos` — a identidade de quem usou é resolvida no coletor, pelo usuário logado no SO.
- **Processamento:** coletor grava a captura → o pipeline roda o motor (regras + detectores + fingerprint) → SQLite → `dashboard.json` → painel. Legado e presente passam pelo **mesmo motor**.

### Retroalimentação — três laços

↺ **LLM → IA clássica.** O que o LLM resolve (uma classificação/rastreio que as regras não cobriam) vira padrão candidato de volta para a camada clássica — menos LLM ao longo do tempo.

↺ **Inventário → motor.** Aprovar ou reprovar uma ferramenta **reclassifica as capturas dela** pelo mesmo motor na regeneração seguinte (ex.: aprovar o ChatGPT leva suas capturas de "Crítico R02" a "Alto R10").

↺ **Auditoria → política.** Os achados realimentam a política — a lacuna do achado A-06 propõe a regra **R15**, fechando a combinação que nenhuma das 14 regras cobria.

---

Segurança e privacidade

## Por desenho, não por promessa

#### Processamento na borda

Detectores e fingerprint rodam na máquina; a camada de metadado nunca grava o valor do dado (CPF etc.) — só tipo e contagem. Prévia mascarada.

#### Identidade confiável

Quem usou é o usuário logado no SO, resolvido no coletor — a página nunca decide a identidade.

#### Isolamento on-premise

Núcleo em VLAN sem saída; só a DMZ fala com fora. Dados da empresa não saem do perímetro.

#### LGPD embutida

A solução detecta dado pessoal em IA pública e marca a obrigação (resposta de privacidade) — o achado A-02 mostra onde isso some hoje.

**MITM é opt-in e avisado**

A interceptação TLS exige aval de Segurança, CA corporativa e aviso aos usuários; documentada com seus riscos (pinning, escopo total).

**Serviço travado por senha**

O agente só é pausado/desinstalado com senha (PBKDF2-SHA256); usuários comuns não o param.

Limitações técnicas

## O que já apareceu — e o que fizemos

Construir o V1 revelou limites reais. Dizê-los é parte da honestidade de engenharia:

| Limitação encontrada | Situação |
|---|---|
| Sensor de rede vê conexão, não conteúdo (tráfego TLS) | Por desenho; conteúdo fica com a extensão. MITM é a alternativa opt-in |
| DNS reverso não resolve em CDN (Cloudflare/Google) | **Resolvido** — índice de IPs + cache de DNS como autoridade |
| Falso-positivo de IP de CDN compartilhado (uma busca virava 11 IAs) | **Resolvido** — cache de DNS filtra o ruído |
| Fingerprint não detecta paráfrase; varredura linear | Pega recorte/colagem; em escala vira índice LSH (mesma matemática) |
| Paridade de hash FNV-1a (JS) × blake2b (Python) | Pendente (fase 2); não afeta o motor de regras |
| Extração de conteúdo por site (APIs privadas mudam) | Best-effort por ferramenta, com fallback genérico |
| Atritos de campo no Windows (alias da Store, execution policy, ACL pt-BR, caminho de rede) | **Resolvidos** no instalador durante os testes |
| Identidade exige usuário logado (serviço sobe no boot antes do login) | Atribui ao usuário assim que alguém loga |

As 6 perguntas da Semana 3

## Respondidas, uma a uma

01

## Como a solução funciona?

Três camadas de captura na borda (extensão, agente e, opcional, proxy) observam o uso de IA na estação e enviam ao coletor central on-premise, que resolve a identidade no SO e dispara o núcleo. O núcleo aplica a IA clássica (regras R01–R14 com precedência, detectores e fingerprint), recorre ao LLM em cascata quando a regra não basta, grava tudo em SQLite e regenera o painel. Legado (planilha) e presente (capturas) passam pelo mesmo motor; o painel dá busca reversa, alertas com evidência e indicadores recalculados.

02

## Quais funções já podem ser demonstradas?

Todas as centrais, ao vivo: a busca reversa da informação (Rastrear), o motor de risco com cadeia de evidência, a Auditoria (indicadores recalculados + os seis achados), a Validação com **15/15** casos do gabarito, os Alertas com filtros, e a captura ao vivo nas três camadas — já com a origem (extensão/agente/proxy) marcada em cada registro. Os dois motores (Python e navegador) têm paridade testada.

03

## Qual modelo será utilizado e por quê?

Duas camadas: a **IA clássica** (motor de regras + detectores determinísticos + fingerprint MinHash/SimHash) como decisor principal, e o **LLM Claude (claude-opus-5)** em cascata para linguagem natural. A escolha vem da tensão do problema: governança precisa de decisão **auditável** (regras), mas a realidade é linguagem natural (LLM). A cascata entrega as duas coisas, e o aprendizado de volta reduz o custo do LLM com o tempo.

04

### De onde vêm os dados?

Dois conjuntos. O **legado**: a base oficial do desafio (650 eventos, 160 alertas, 8 ferramentas, 20 tipos, 72 usuários, 14 regras, 12 abas; classificação pronta; fictícios). E o **presente**: as capturas ao vivo das estações, com identidade resolvida no SO e origem marcada. Os dois entram na mesma base e no mesmo motor.

05

### Como os dados serão utilizados e retroalimentados?

São usados para reaplicar a política e produzir risco, alertas e indicadores com evidência. A retroalimentação tem três laços: o **LLM ensina a IA clássica** (o que ele resolve vira padrão); o **inventário reclassifica** as capturas de uma ferramenta quando seu status muda (aprovar/reprovar); e a **auditoria realimenta a política** (o achado A-06 propõe a regra R15). Ou seja, o sistema aprende com o uso, com a decisão do admin e com a própria auditoria.

06

### Quais limitações técnicas já apareceram?

Várias, e quase todas já tratadas (tabela acima): o sensor de rede vê metadado e não conteúdo (TLS) — por desenho; o DNS reverso não resolvia em CDN e o IP compartilhado gerava falso-positivo — resolvidos com índice de IPs + cache de DNS; o fingerprint não pega paráfrase e a varredura é linear — some em escala com LSH; a paridade de hash FNV-1a × blake2b segue pendente (fase 2, não afeta o motor); e os atritos de implantação no Windows (alias da Store, execution policy, ACL em pt-BR, caminho de rede) foram corrigidos no instalador durante os próprios testes.

---

Critério da Semana 3

## Arquitetura, dados e uso da IA

01

### Modelo de IA e justificativa

IA clássica (decisor auditável) + LLM claude-opus-5 em cascata (linguagem natural), com aprendizado de volta. Auditável e econômico.

02

### Arquitetura da solução

Borda (3 camadas) → coletor → núcleo (pipeline + motor) → base unificada → painel; on-premise com VLAN isolada e DMZ de notificação.

03

### Fluxo de dados e retroalimentação

Legado + presente no mesmo motor; três laços de retroalimentação (LLM→clássica, inventário→motor, auditoria→política).

04

### Protótipo V1 e função principal

Roda de verdade (15/15, 98,8%); função principal — a busca reversa da informação — demonstrável ao vivo.

---